

Chapter 3.3

State Management and Navigation principles

Contents

State Management

Navigation Principles

Navigation

- Tab

- Drawer

- Dialog

- Picker

- Snackbar

State Management

State Management

State management involves managing the data that changes over time and triggers UI updates.

State of an app is everything that exists in memory when the app is running.

This includes the app's assets, all the variables that the Flutter framework keeps about the UI, animation state, textures, fonts, and so on.

State Management

Ephemeral State

Also called UI state or local state.

It is the state you can neatly contain in a single widget.

App State

Also called shared state

UI state that you want to share across many parts of your app, and that you want to keep between user sessions

State Management

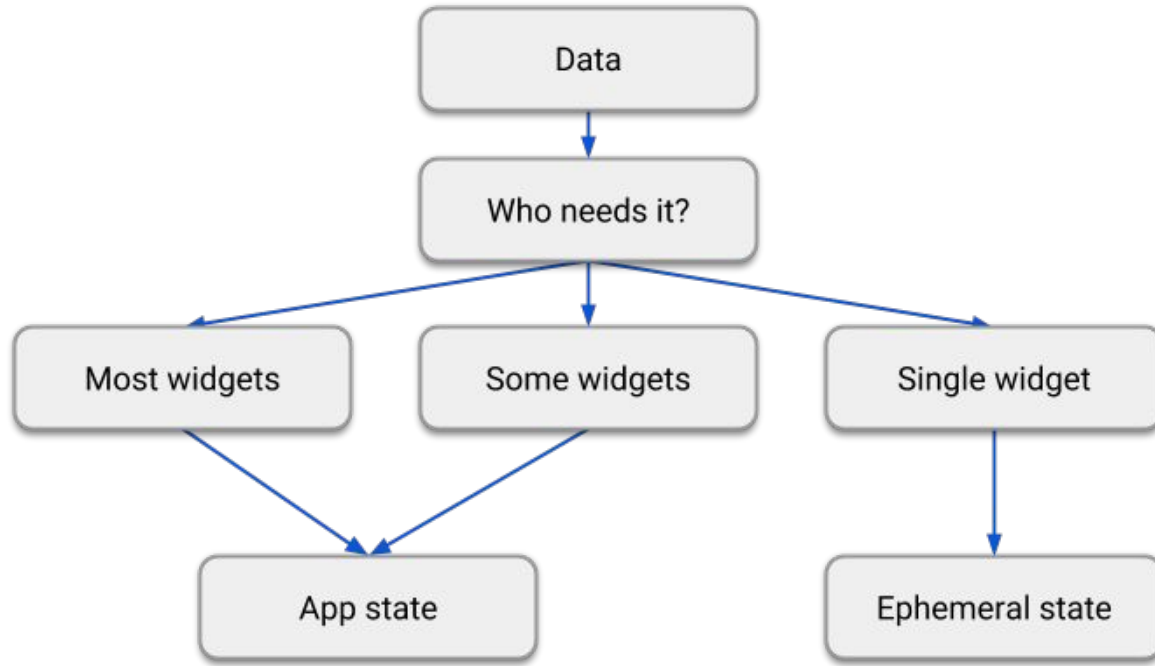
Examples of Ephemeral State:

- current page in a `PageView`
- current progress of a complex animation
- current selected tab in a `BottomNavigationBar`

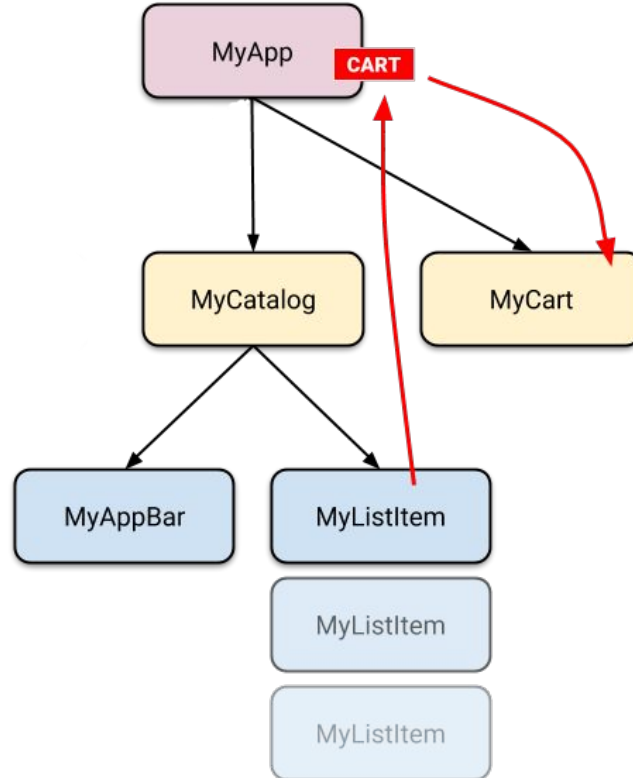
Examples of Application State:

- user preferences
- login info
- notifications in a social networking app
- the shopping cart in an e-commerce app
- read/unread state of articles in a news app

State Management



State Management



State Management - Provider

Provides a simple way to share state across multiple widgets.

Uses `ChangeNotifier` to notify listeners when the state changes.

dependencies:

flutter:

 sdk: flutter

provider:version

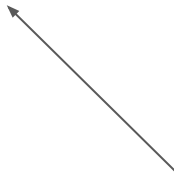
State Management - Provider

Insert dependency:

```
import 'package:provider/provider.dart';
```

State Management - Provider

```
class CounterProvider extends ChangeNotifier {  
    int _counter = 0;  
    int get counter => _counter;  
    void increment() {  
        _counter++;  
        notifyListeners();  
    }  
}
```



It notify listeners when
the state changes

State Management - Provider

```
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (context) => CounterProvider(),  
      child: someWidget,  
    ),  
  );  
}
```

State Management - Provider

Widget with multiple providers:

```
MultiProvider(  
  providers: [  
    ChangeNotifierProvider(create: (context) => Provider1()),  
    ChangeNotifierProvider(create: (context) => Provider2()),  
  ],  
  child: somewidget,  
),
```

State Management - Provider

```
body: Center(  
  child: Consumer<CounterProvider>(←  
    builder: (context, counter, child) {  
      return Text($counter.counter,);  
    },  
  ),  
,  
  
floatingActionButton: FloatingActionButton(  
  onPressed: () {  
    context.read<CounterProvider>().increment();  
  },  
  tooltip: 'Increment',  
  child: const Icon(Icons.add),  
)
```



Consumer widget

Navigation principles

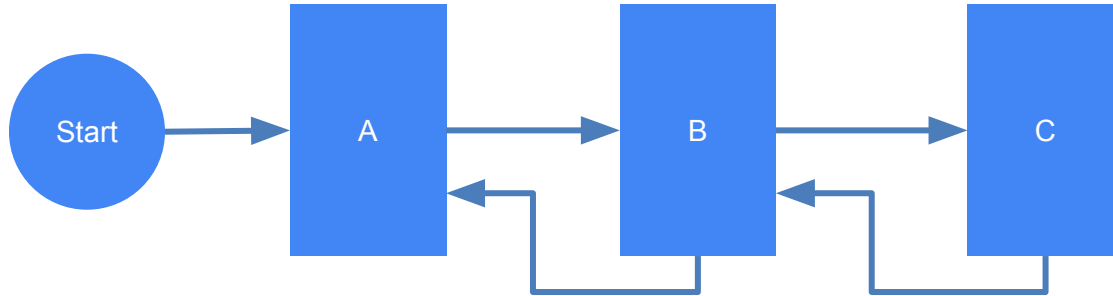
Navigation Principles

1. Fixed start destination
2. Navigation state is represented as a stack of destinations
3. Up and Back are identical within your app's task
4. The Up button never exits your app
5. Deep linking simulates manual navigation

Navigation Principles

1. Fixed start destination

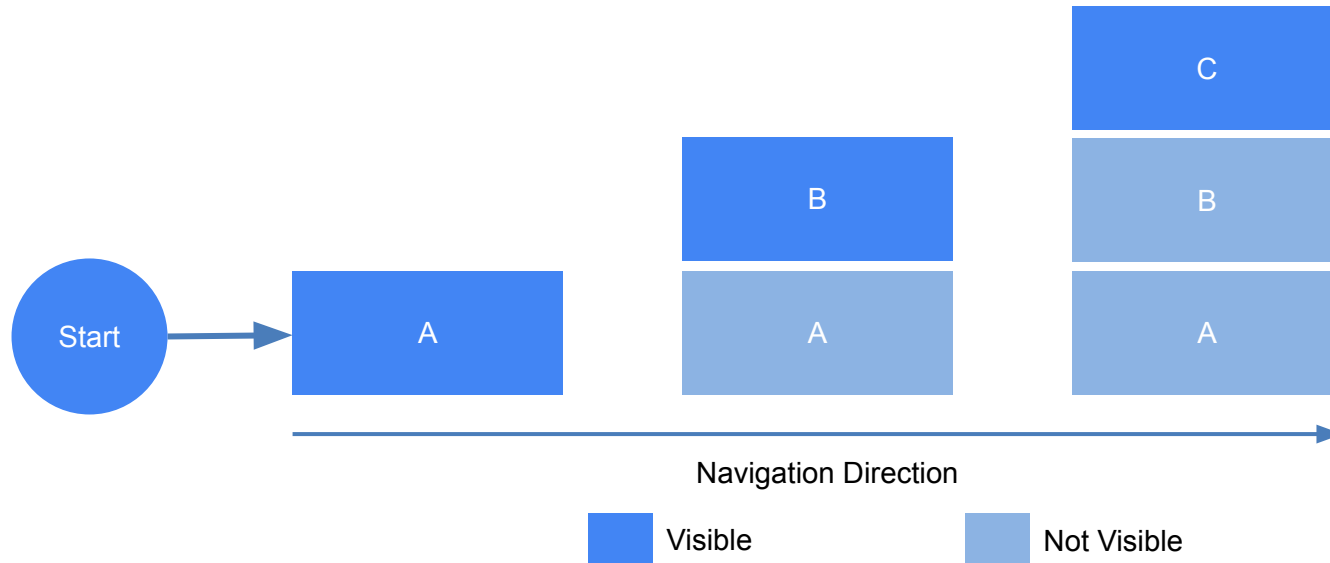
Every app you build has a fixed start destination. This destination is also the last screen the user sees when they return to the launcher after pressing the Back button.



Navigation Principles

2. Navigation state is represented as a stack of destinations

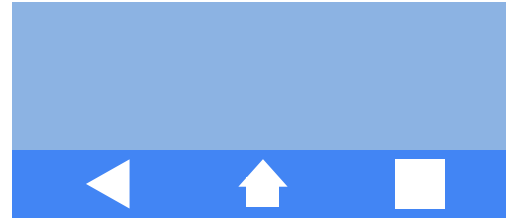
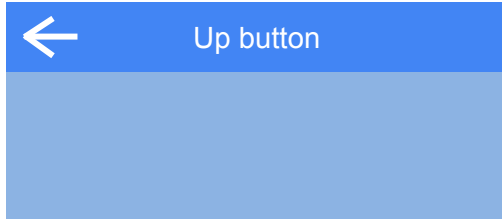
The top of the stack is the current screen, and the previous destinations in the stack represent the history of where you've been.



Navigation Principles

3. Up and Back are identical within your app's task

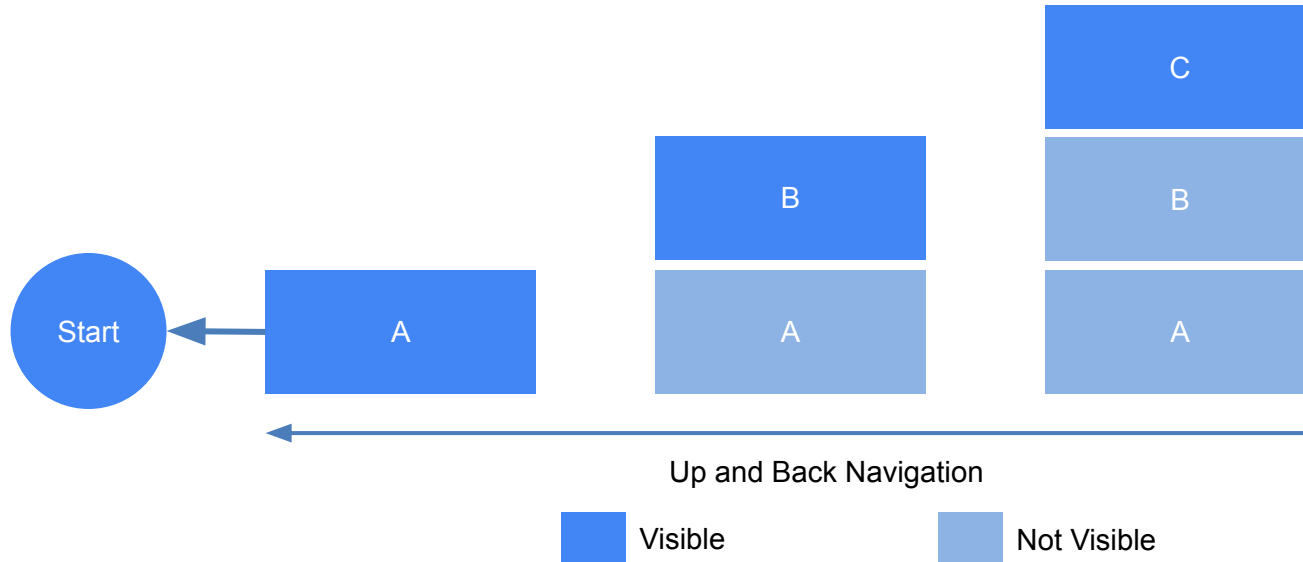
The Up button appears in the app bar at the top of the screen. Within your app's task, the Up and Back buttons behave identically.



Navigation Principles

3. Up and Back are identical within your app's task

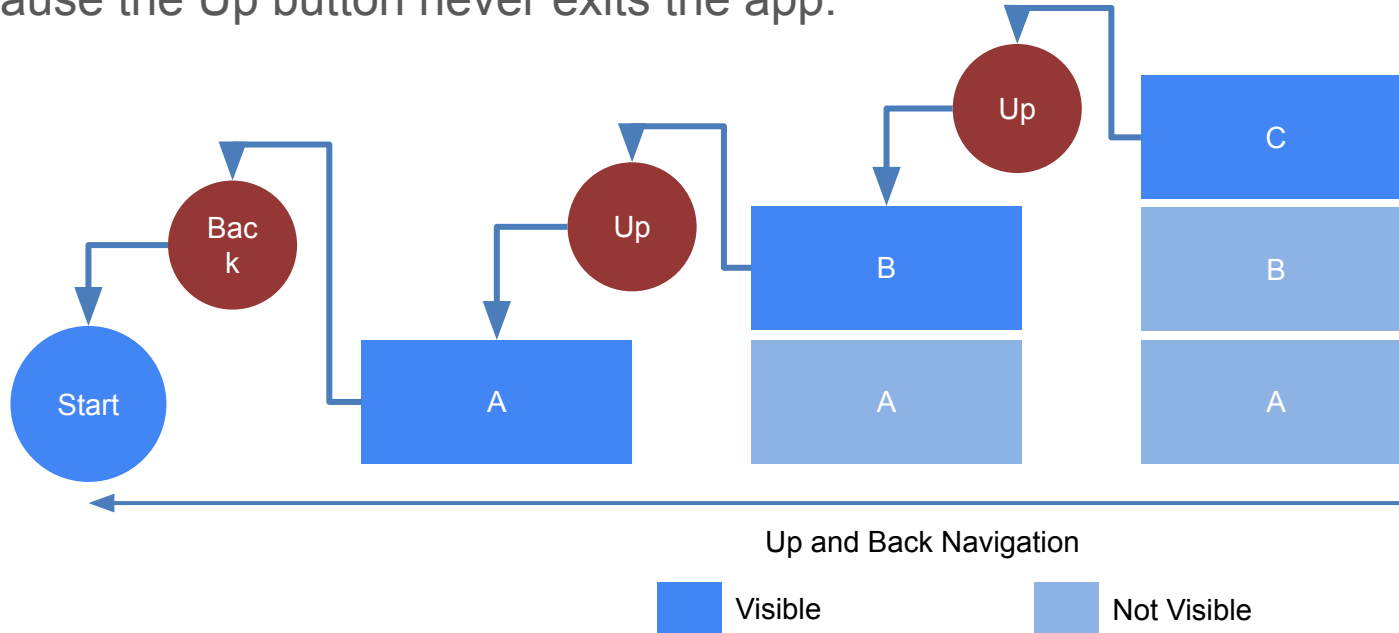
When you press the Back button, the current destination is popped off the top of the back stack, and you then navigate to the previous destination.



Navigation Principles

4. The Up button never exits your app

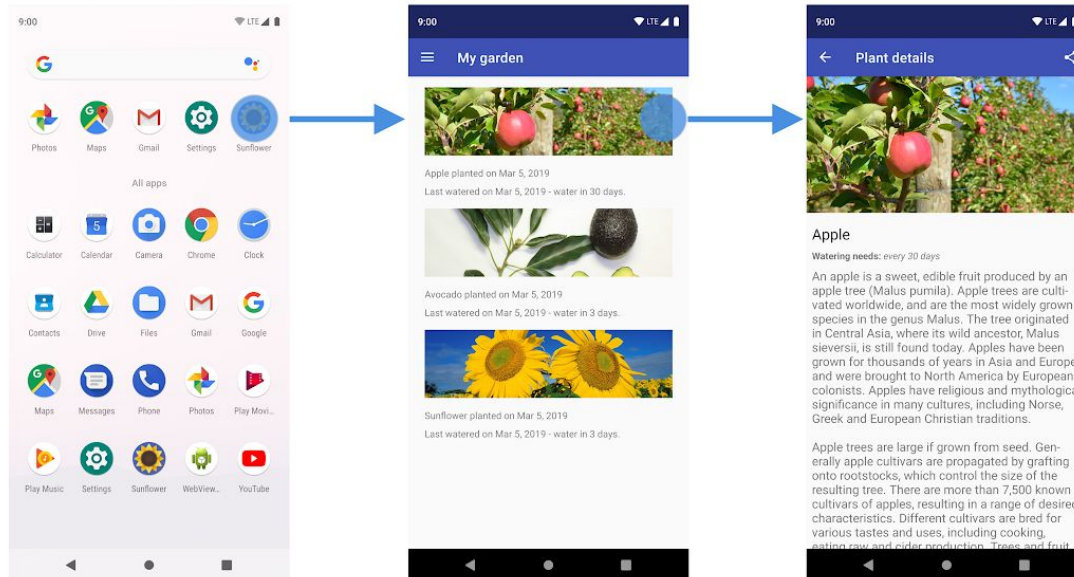
If a user is at the app's start destination, then the Up button does not appear, because the Up button never exits the app.



Navigation Principles

5. Deep linking simulates manual navigation

Navigation design should simulate manual navigation. E.g. From the app navigator to the main screen of an app, and then to a detail screen.



Tab

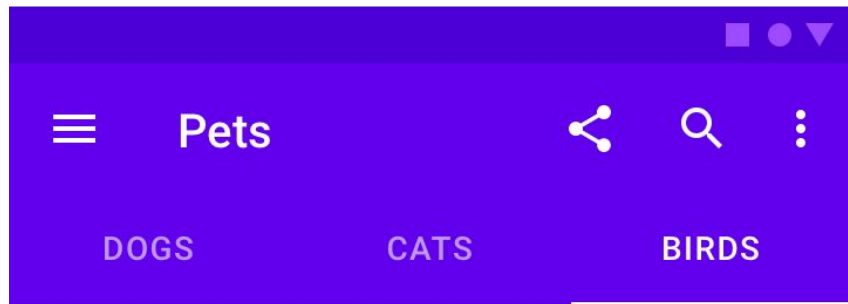
Tab

A `TabBar` and `TabBarView` combination is commonly used to create tabbed interfaces in Flutter.

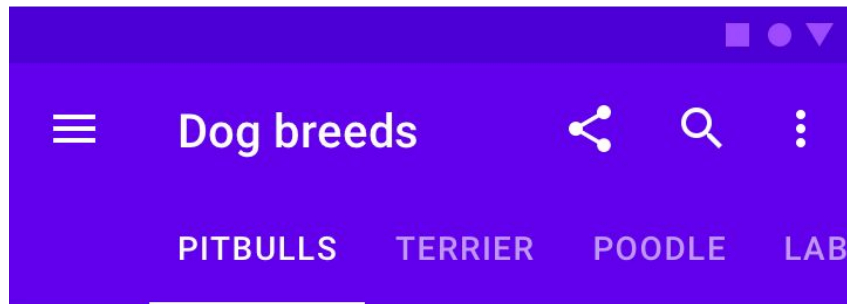
The `TabBar` displays the tabs, while the `TabBarView` displays the content associated with each tab.

The `TabController` manages the state of the tabs, allowing you to switch between them.

Tab



1. Fixed Tabs



2. Scrollable Tabs

Tab

Step 1: Create a TabController

```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(),  
  ),  
);
```

Tab

Step 2: Create the tabs

```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      appBar: AppBar(  
        bottom: const TabBar(  
          tabs: [  
            Tab(icon: Icon(Icons.directions_car)),  
            Tab(icon: Icon(Icons.directions_transit)),  
            Tab(icon: Icon(Icons.directions_bike)),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

Tab

Step 2: Create content for each tab

```
body: const TabBarView(  
  children: [  
    Icon(Icons.directions_car),  
    Icon(Icons.directions_transit),  
    Icon(Icons.directions_bike),  
  ],  
),
```

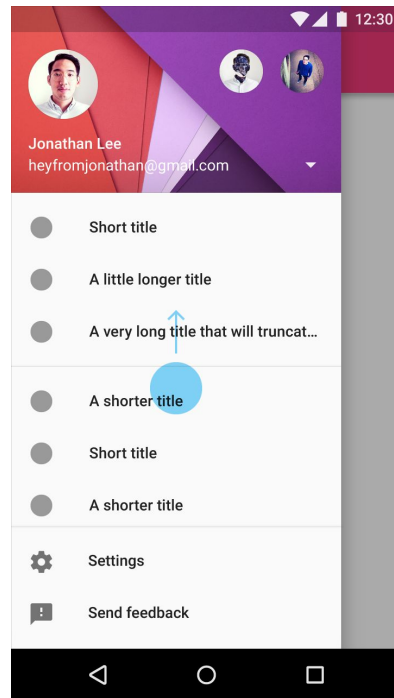
Drawer

Drawer

In Flutter, use the Drawer widget in combination with a Scaffold to create a layout with a Material Design drawer.

This recipe uses the following steps:

1. Create a Scaffold.
2. Add a drawer.
3. Populate the drawer with items.
4. Close the drawer programmatically.

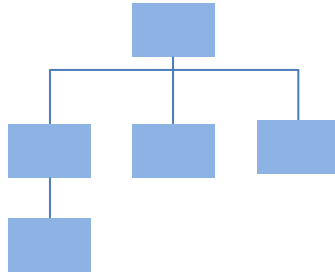


Navigation Drawer

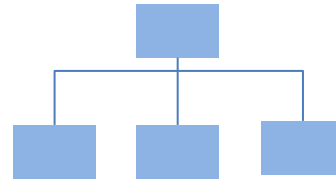
Recommended for:



≥ 5 top-level destinations



≥ 2 levels of navigation hierarchy



Quick navigation between unrelated destinations

Drawer

Step 1: Create a Scaffold

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar without hamburger button'),  
  ),  
  
);
```


Drawer

Step 2: Add a drawer

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar with hamburger button'),  
  ),  
  drawer: Drawer(  
    child: // Populate the Drawer in the next step.  
  ),  
);
```

Drawer

Step 3: Populate the drawer with items

```
Drawer(  
  // Add a ListView to the drawer.  
  child: ListView(  
    // Important: Remove any padding from the ListView.  
    padding: EdgeInsets.zero,  
    children: [  
      const DrawerHeader(  
        decoration: BoxDecoration(color: Colors.blue,),child: Text('Drawer Header'),),  
      ListTile(title: const Text('Item 1'), onTap: () { ... },),  
      ListTile(title: const Text('Item 2'), onTap: () { ... },),  
    ],  
  ),//End of ListView  
)//End of Drawer
```

Drawer

Step 4: Open the drawer programmatically.

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar with hamburger button'),  
    leading: Builder(  
      builder: (context) {  
        return IconButton(  
          icon: const Icon(Icons.menu),  
          onPressed: () { Scaffold.of(context).openDrawer(); },  
        );  
      },  
    ),  
  ),  
  drawer: Drawer(  
    child: // Populate the Drawer in the last step.  
  ),  
);
```

Drawer

Step 5: Close the drawer programmatically

```
ListTile(  
  title: const Text('Item 1'),  
  onTap: () {  
    // Update the state of the app  
    // ...  
    // Then close the drawer  
    Navigator.pop(context);  
  },  
)
```

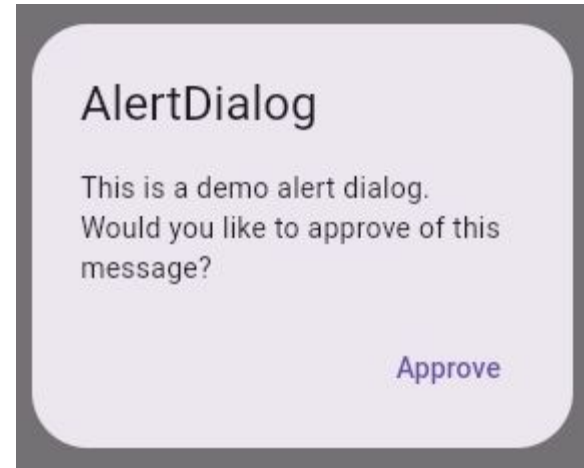
Dialog

AlertDialog

An alert dialog (also known as a basic dialog) informs the **user** about situations that require **acknowledgment**.

An alert dialog has an optional title and an optional list of actions.

The title is displayed above the content and the actions are displayed below the content.

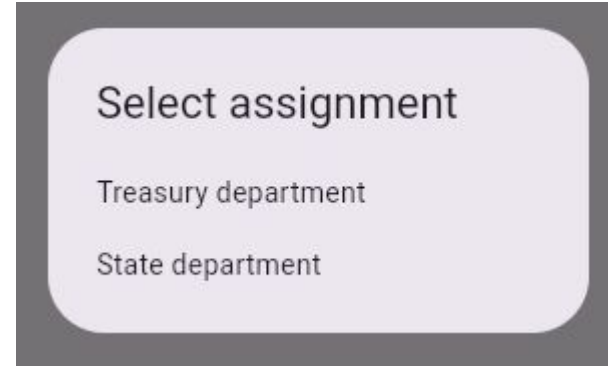


```
Future<void> _showMyDialog() async {  
  return showDialog<void>(  
    context: context, barrierDismissible: false, // user must tap button!  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('AlertDialog Title'),  
        content: const SingleChildScrollView(  
          child: ListBody(  
            children: <Widget>[Text('Would you like to approve of this message?'), ],  
          ),  
        ),  
        actions: <Widget>[  
          TextButton( child: const Text('Approve'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ),  
        ],  
      ),  
    ),  
  );  
}
```

SimpleDialog

A simple dialog offers the **user** a **choice** between several options.

A simple dialog has an optional title that is displayed above the choices.



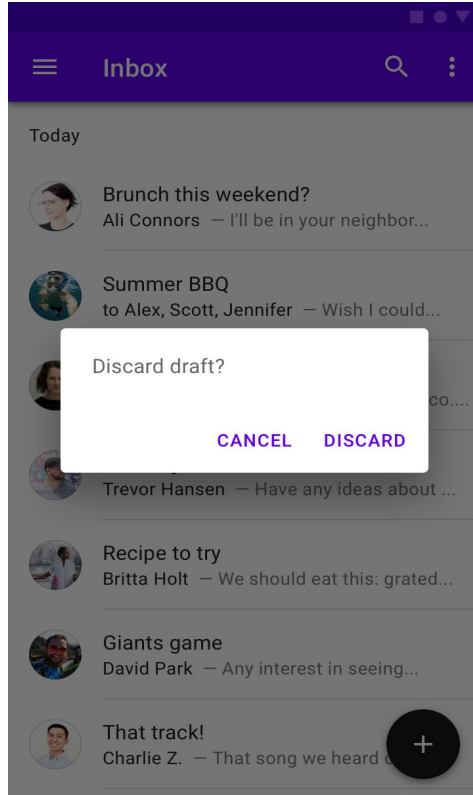

```

Future<void> _askedToLead() async {
  switch (await showDialog<Department>(
    context: context,
    builder: (BuildContext context) {
      return SimpleDialog(
        title: const Text('Select assignment'),
        children: <Widget>[
          SimpleDialogOption(
            onPressed: () { Navigator.pop(context, Department.treasury); },
            child: const Text('Treasury department'),
          ),
          SimpleDialogOption(
            onPressed: () { Navigator.pop(context, Department.state); },
            child: const Text('State department'),
          ),
        ],
      );
    }
  )) {
    case Department.treasury:
      // Let's go.
      // ...

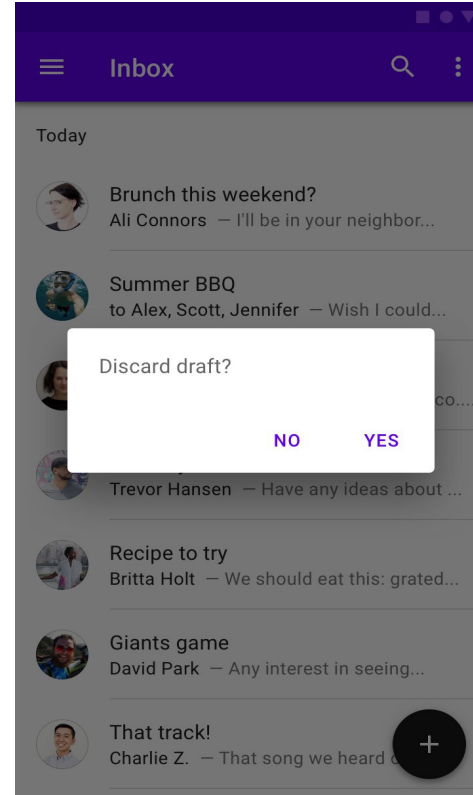
```

Which AlertDialog is better? Why?

(A)



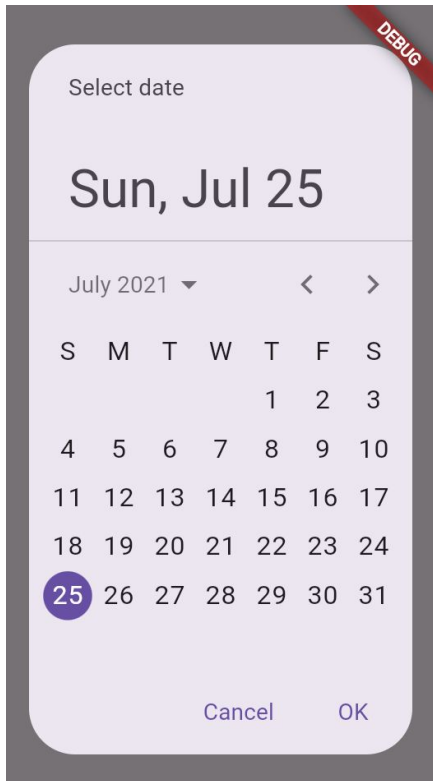
(B)



Date and Time Pickers

Using `showDatePicker` and `showTimePicker`.

Using these pickers helps ensure that your users can pick a time or date that is valid, formatted correctly, and adjusted to the user's locale.



Date and Time Pickers

```
DateTime _selectedDate = DateTime.now();
```

```
Future<void> _selectDate(BuildContext context) async {  
  final DateTime? picked = await showDatePicker(  
    context: context,  
    initialDate: _selectedDate,  
    firstDate: DateTime(2000),  
    lastDate: DateTime(2100),  
  );  
  if (picked != null) {  
    setState(() {  
      _selectedDate = picked;  
    });  
  }  
}
```



Date and Time Pickers

```
OutlinedButton(  
  onPressed: () {  
    _selectDate(context);  
  },  
  child: Text('$_selectedDate'),  
),
```

2024-12-03 11:21:39.006

Snackbar

Inform your users when certain actions take place.

For example, a message has been deleted.

Give the user an option to undo an action.

Snackbar

Insert a snackbar within the Scaffold.

```
final snackBar = SnackBar(  
  content: const Text('Yay! A SnackBar!'),  
  action: SnackBarAction(  
    label: 'Undo',  
    onPressed: () {  
      // Some code to undo the change.  
    },  
  ),  
);
```

```
// Find the ScaffoldMessenger in the widget tree and use it to show a SnackBar.  
ScaffoldMessenger.of(context).showSnackBar(snackBar);
```

Snackbar

Component	Priority	User Action
Snackbar	Low	Optional It disappears automatically
Dialog	High	Required It blocks app usage until the user takes a dialog action or exists the dialog

Find Out More

Tab, <https://docs.flutter.dev/cookbook/design/tabs>

Drawer, <https://docs.flutter.dev/cookbook/design/drawer>

AlertDialog, <https://api.flutter.dev/flutter/material/AlertDialog-class.html>

SimpleDialog, <https://api.flutter.dev/flutter/material/SimpleDialog-class.html>

Snackbar, <https://docs.flutter.dev/cookbook/design/snackbars>